

FORCESEEK Hints

#sql-server #performance #hints #optimizer

What FORCESEEK Does

The FORCESEEK table hint tells the optimizer: “You must use an index seek to access this table. Do not scan.”

```
Select i.Id  
From ImgImage i With (NoLock)  
Join ImgRxBgAssoc IR With (NoLock, FORCESEEK) On i.Id = IR.ImgId
```

The optimizer will find an index on ImgRxBgAssoc that supports a seek on ImgId and use it. If no such index exists, the query fails with an error rather than falling back to a scan.

When FORCESEEK Appears in Code

FORCESEEK is almost always a band-aid. Its presence tells a story:

1. Someone observed a slow query with a scan on this table.
2. They forced a seek and performance improved.
3. The underlying reason for the scan was never addressed.

The scan usually happens because of:

- **LEFT JOIN OR Anti-Pattern**: OR conditions prevent the optimizer from choosing seeks.
- **Non-SARGable Predicates**: function-wrapped columns can't seek.
- **UNION ALL Views**: the optimizer can't push predicates through the view cleanly.
- Bad cardinality estimates. The optimizer thinks a scan is cheaper because it overestimates the result size.
- Stale statistics. The optimizer's row estimates are based on outdated data.

Why It's a Band-Aid

FORCESEEK addresses the symptom (scan instead of seek) but not the cause (why the optimizer chose a scan). Problems with this approach:

1. It Overrides the Optimizer's Judgment

Sometimes the optimizer chooses a scan because it IS the better plan. If the table is small or the query returns a large percentage of rows, a scan plus filter is faster than thousands of individual seeks. FORCESEEK removes this flexibility.

2. It Masks the Real Problem

The presence of FORCESEEK should trigger investigation: why doesn't the optimizer choose a seek naturally? Fixing the root cause benefits all queries on the table, not just the one with the hint.

3. It Creates Fragile Plans

If the index that supports the seek is dropped, renamed, or reorganized, the query fails outright instead of degrading gracefully to a scan.

4. It Doesn't Compose

FORCESEEK forces a seek on one table, but if the surrounding query structure is broken (LEFT JOIN + OR, for example), the optimizer still produces a suboptimal plan for everything else.

The Right Approach

1. See FORCESEEK, then investigate why the scan happens.
2. Fix the root cause (refactor the query pattern, fix the predicates, update statistics).
3. Remove the FORCESEEK and verify the optimizer chooses seeks naturally.
4. If the optimizer still scans after the root cause fix, then keep the hint. Now it's a deliberate choice, not a band-aid.

When to Keep FORCESEEK

- Parameterized queries with extreme cardinality variance. The optimizer might choose a scan based on the average case, but seeks are better for 99% of executions. If you can't use **local variables** or **OPTION (RECOMPILE)**, FORCESEEK is a reasonable backstop.
- Known stale statistics. If you can't update statistics frequently enough and the optimizer makes wrong choices, FORCESEEK stabilizes the plan.
- Transitional. Keep it during a phased refactoring to prevent regressions, then remove it once the full fix is in place.

Real-World Example

In `lsp_ImgGetListOfTopXImagesToMove` (v6), both `ImgRxImgAssoc` and `ImgCanImgAssoc` had FORCESEEK hints. These were added because the **LEFT JOIN OR Anti-Pattern** caused the optimizer to scan these association tables. The real fix turned out to be the materialize-once + EXISTS refactoring (Fix #2 in **LEFT JOIN OR Anti-Pattern**), which converted the LEFT JOINS to clean INNER JOINS inside EXISTS clauses. After the refactor, the hints were redundant: the optimizer picked seeks naturally on each association table.

Related Concepts

- **LEFT JOIN OR Anti-Pattern**: the most common root cause when you see FORCESEEK.
- **Non-SARGable Predicates**: another reason the optimizer can't seek.
- **UNION ALL Views**: predicate pushdown failures through views cause unnecessary scans.
- **Query Store Triage**: look for plans with high reads that might have been "fixed" with FORCESEEK.